

L0 Simulation Report

PIBT on Abstract Grids

1 Context

As part of a study on deploying DotBot robots in an intralogistics environment, a Python implementation of the PIBT (*Priority Inheritance with Backtracking*) algorithm was developed to coordinate the movement of multiple agents on a discrete grid. Before any real-hardware testing, a purely algorithmic validation level is required to establish the theoretical performance ceiling of the planner, independent of any physical constraints. The purpose of this validation part is to confirm that failures observed on physical robots origin is not the planning algorithm.

2 Reference: Okumura et al. (2022)

Okumura et al. published the PIBT algorithm in 2022 in *Artificial Intelligence* (vol. 310). Their reference implementation, written in C++, guarantees *reachability* meaning every agent reaches its goal within finite time — provided the navigation graph is biconnected, a property satisfied by any obstacle-free rectangular grid. In their experiments on an empty 8×8 grid (64 cells), they report a success rate of 96 % at $N = 40$ agents, 84 % at $N = 50$, and paradoxically 100 % at both $N = 60$ and $N = 64$: at very high density, agents engage in simultaneous circular rotations that allow progress even when every cell is occupied. Planning time remains below 4 ms per step even at $N = 64$, on an ordinary laptop.

3 Results

The Python implementation was evaluated on the three target grid resolutions (4×4 , 5×5 , 8×8), sweeping N from 2 up to the full grid capacity, over 30 independent random seeds and a step cap of 100. The results on the 5×5 grid are presented below as a representative sample.

These results are consistent with the Okumura reference: at low density ($\rho < 0.25$), success stays above 98 % and the detour ratio stays near 1, indicating that agents follow near-optimal paths. The gradual degradation as ρ increases reflects the emergence of longer priority-inheritance chains and livelock situations that the 100-step cap does not always resolve — whereas Okumura uses a cap of 1 000. The inflection at $\rho = 0.60$ ($N = 15$) marks the efficiency threshold identified in Figure 1: beyond this point both cost per agent and makespan accelerate sharply, a behaviour common to all three grid resolutions tested. Planning time, below 0.3 ms per step even at $N = 25$, confirms that the algorithmic complexity $\mathcal{O}(|A| \cdot (\Delta(G) + \log |A|))$ is preserved, with Python’s interpreter introducing only a constant overhead that does not affect the algorithm’s behaviour.

Table 1: PIBT performance on a 5×5 grid (25 cells, 400 mm cells, 30 seeds, step cap = 100).

N	ρ	Sum-of-costs	Mean makespan	Detour ratio	Planning time (ms/step)
2	0.08	6.6	4.4	1.07	0.013
5	0.20	18.2	6.0	1.10	0.032
10	0.40	33.7	6.9	1.22	0.070
15	0.60	60.2	9.1	1.46	0.116
20	0.80	101.7	14.6	1.99	0.180
25	1.00	250.7	27.3	3.41	0.294

Cost per agent and mean makespan vs. occupancy ρ — both metrics accelerate past $\rho = 0.6$ (5×5 grid, 400 mm cells, 30 seeds)

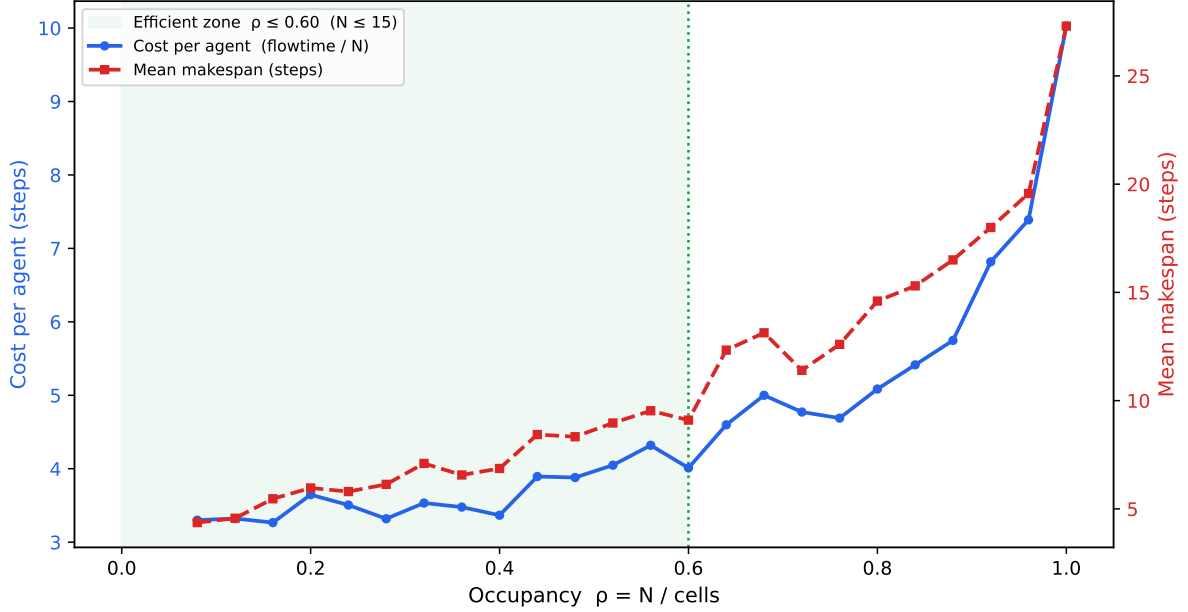


Figure 1: Cost per agent (flowtime/ N) and mean makespan as a function of occupancy ρ on a 5×5 grid (400 mm cells, 30 seeds, step cap = 100). Both metrics accelerate sharply past $\rho \approx 0.60$, marking the efficiency threshold at $N = 15$ agents.

4 Outlook: from simulation to physical deployment

Across all three grid resolutions, both the cost per agent and the mean makespan remain near-linear up to an occupancy of roughly $\rho = 0.60$, then accelerate sharply beyond that point. This threshold corresponds to the following agent counts on the target $2 \text{ m} \times 2 \text{ m}$ floor area:

- 4×4 grid (500 mm cells, 16 cells): $\lfloor 0.60 \times 16 \rfloor = 10$ agents,
- 5×5 grid (400 mm cells, 25 cells): $0.60 \times 25 = 15$ agents,
- 8×8 grid (250 mm cells, 64 cells): $\lfloor 0.60 \times 64 \rfloor = 38$ agents.

The algorithm therefore predicts that, for the chosen floor area, operating below 60% occupancy keeps individual robot costs proportional to distance — beyond that, priority-inheritance chains grow fast enough to degrade throughput non-linearly. The next question

is whether this threshold holds when the agents are physical DotBot robots subject to communication latency, wheel slip, and discrete navigation steps. The following validation levels (L1 onward) are designed to test exactly this: by running the same PIBT planner on real DotBots and comparing the empirical cost curves against the L0 baseline, we can isolate how much of the degradation (if any) originates from hardware constraints rather than from the planning algorithm itself.

5 Level 1 — Real DotBot Hardware

Where L0 established the *algorithmic* ceiling on abstract grids, Level 1 measures the *physical* limit on real DotBots, where the threshold between the robot’s turning footprint and the cell size actually bites. The same PIBT planner drives the hardware through a step-by-step barrier (100 mm arrival threshold, 8 s per-step timeout): the planner advances one synchronous step, the bots are commanded to their next cells, and the loop waits until every bot is within threshold of its target before re-synchronising from the LH2 localisation. The headline quantity is the **plan-to-real gap** $\Delta = \text{success}_{L0} - \text{success}_{L1}$, which isolates how much of any degradation comes from hardware rather than from the algorithm.

The batch harness (`test_real_dotbot_pibt_*bots_*runs.py`, with the shared logger `l1_metrics.py`) records one row per run to `l1_per_run.csv` and the per-(grid, N) aggregate to `l1_summary.csv`, using columns aligned with the L0 results CSV so the two levels join directly.

5.1 Metrics

- **Success rate** — bots reaching their goal cell over N , averaged over seeds (mean $\pm$ 95% CI).
- **Overshoot** — real stop distance to the target versus the 100 mm threshold; the direct measurement of the turning limit.
- **Step-timeout rate** — fraction of (bot $\times$ step) that did not settle within the 8 s barrier.
- **Throughput** — goals completed per minute (the intralogistics framing).

5.2 Results

[pending hardware run — to be filled from `l1_summary.csv` once the 4×4 (500 mm), 5×5 (400 mm) and 8×8 (250 mm) sweeps have been run on real DotBots, then overlaid against the L0 baseline for the plan-to-real gap.]